МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Курсовая работа
по курсу «Программирование графических процессоров»

Обратная трассировка лучей (Ray Tracing) на GPU (вариант 9: гексаэдр, додекаэдр, икосаэдр)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы — использование GPU для создания фотореалистической визуализации: рендеринг полузеркальных и полупрозрачных правильных геометрических тел, получение эффекта бесконечности, создание анимации.

Сцена. Прямоугольная текстурированная поверхность (пол), над которой расположены три платоновых тела — по варианту 9: гексаэдр, додекаэдр и икосаэдр. Сверху находятся несколько источников света. На каждом ребре многогранника располагается определенное количество точечных источников света. Грани тел обладают зеркальным и прозрачным эффектом; за счет многократного переотражения лучей внутри тела возникает эффект бесконечности.

Камера. Камера выполняет облет сцены: положение и точка направления задаются в цилиндрических координатах (r, φ, z) и меняются во времени по законам $r(t) = r0 + Ar \cdot \sin(\omega r \cdot t + pr)$, $z(t) = z0 + Az \cdot \sin(\omega z \cdot t + pz)$, $\varphi(t) = \varphi 0 + \omega\varphi \cdot t$, где $t \in [0, 2\pi]$; аналогичные формулы с независимыми параметрами задают точку наблюдения.

Требуется реализовать алгоритм обратной трассировки лучей с использованием технологии CUDA и выполнить покадровый рендеринг сцены. Для устранения эффекта «зубчатости» выполнить сглаживание (например, SSAA). Полученный набор кадров склеить в анимацию любым доступным программным обеспечением. Подобрать параметры сцены, камеры и освещения таким образом, чтобы получить наиболее красочный результат. Провести сравнение производительности GPU и CPU, для чего дополнительно реализовать алгоритм без использования CUDA.

Ввод. Программа принимает на вход: количество кадров; путь к выходным изображениям со спецификатором %d (формат изображений — как в лабораторной работе 2); разрешение кадра и угол обзора в градусах по горизонтали; параметры движения камеры и точки наблюдения (по 10 чисел); параметры тел — центр, цвет, радиус описанной сферы, коэффициент отражения, коэффициент прозрачности, количество точечных источников света на ребре; параметры пола — четыре точки, путь к текстуре, оттенок цвета и коэффициент отражения; количество (не более четырех) и параметры источников света — положение и цвет; максимальную глубину рекурсии и квадратный корень из количества лучей на один пиксель (SSAA).

Ключи запуска: --cpu — расчет только на центральном процессоре; --gpu — расчет на видеокарте (используется и при запуске без аргументов); --default — вывод в stdout конфигурации входных данных, при которой получается наиболее красочный результат. В процессе работы программа выводит в stdout статистику: {номер кадра}, время на обработку кадра в миллисекундах и общее количество

лучей (все трассированные лучи, включая теневые). Компиляция: nvcc -Werror cross-execution-space-call -lm.

Работа выполняется на оценку «4»: без рекурсии, с отражениями, упрощенная обработка прозрачности (преломленный луч далее не отражается), упрощенное наложение текстур (пол всегда квадратный и параллелен плоскости XY).

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же запускал программу с разными конфигурациями и готовил кадры для отчета.

Графический процессор — NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS, драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140). Сборку и прогоны всех серий (анимация, каркасный рендер, ракурсы, бенчмарк GPU и CPU) делал скриптом на этой же машине, конвертацию кадров в PNG — скриптом на чистом Python без сторонних пакетов, склейку кадров в видео — ffmpeg.

Метод решения

Ввод-вывод. Программа читает конфигурацию сцены из stdin (формат — из условия), кадры пишет в файлы в формате лабораторной работы 2 (8 байт little-endian — ширина и высота, затем пиксели RGBA), в stdout — построчную статистику. Отладочная информация пишется в stderr; при любой ошибке печатается «ERROR: …», и программа завершается с кодом возврата 0. Дополнительно реализован ключ --wireframe — каркасный рендер (рёбра тел, рамка и сетка пола, источники света) для наглядной проверки геометрии сцены.

Геометрия. Тела — выпуклые многогранники, вписанные в заданную описанную сферу. Плоскости граней строятся на хосте через двойственные тела: направление на центр грани — это вершина двойственного тела (гексаэдр — октаэдр, додекаэдр — икосаэдр, икосаэдр — додекаэдр), а плоскость грани проходит через опорные точки тела в этом направлении (максимум скалярного произведения вершины на направление, умноженный на радиус). Здесь есть тонкость: у пары икосаэдр–додекаэдр существует два вложенных положения двойников, и для получения правильных граней нужна согласованная ориентация вершин — при несогласованной плоскости проходят через одиночные вершины, а не через грани,

и тела выглядят «скругленными». Рёбра — пары ближайших вершин тела; на них равномерно размещаются точечные источники света. Пересечение луча с телом считается клиппингом по плоскостям граней (накопление ближней и дальней границ интервала) с быстрым претестом описывающей сферой; пол — пересечение с плоскостью и проверка попадания в квадрат.

Трассировка. Основная трассировка итеративная, без рекурсии (требование на «4»): на каждом попадании вычисляется освещенность, затем строится под-путь преломления, а луч продолжается зеркальным отражением с затухающим весом; при весе ниже порога трассировка обрывается. Освещенность — фоновая, диффузная и зеркальная (Блинн—Фонг) компоненты по всем источникам; для каждого источника, кроме стоящих на рёбрах собственного тела, выпускается теневой луч: сегмент до источника проверяется клиппингом по телам сцены. Реберные источники светят локально (спад ~$1/d^2$), основные — без спада. Отражения обрабатываются и на телах, и на полу.

Прозрачность. Прозрачность упрощенная (на «4»): преломленный луч далее не порождает зеркальных отражений. Преломление — по закону Снеллиуса (показатель преломления 1.5). Внутри тела возможно полное внутреннее отражение — оно физически происходит без потерь, поэтому вес луча при нем не гасится: луч многократно переотражается внутри тела до выхода, и возникает эффект бесконечности. Достигнув пола, преломленный луч гаснет.

Тон-маппинг. В сцене до четырехсот источников (в основной конфигурации — 396 реберных и 3 основных), и их суммарный вклад — HDR-величина, многократно превышающая единицу. Первый прогон показал пересвет: жесткое ограничение диапазона при записи пикселя оставляло белыми около трети кадра — все, кроме неба. Поэтому в конвейер добавлена стадия тон-маппинга: сначала экспозиция, затем мягкое сжатие по Рейнхарду $c/(1+c)$ по каждому каналу; яркость источников подобрана так, чтобы середина диапазона приходилась на типичную освещенность сцены (реберные источники взяты с коэффициентом 0.1 и локальным спадом, основные — приглушены). Это устранило пересвет и сохранило цвет и детали в ярких областях.

Сглаживание и текстуры. Сглаживание — SSAA: n×n лучей на пиксель с равномерным распределением по площади пикселя. Текстура пола фильтруется билинейно; один и тот же код используется в версиях для CPU и GPU, поэтому результаты обеих версий совпадают. Камера движется по формулам условия, угол обзора задается по горизонтали; при почти вертикальном взгляде, когда ось z нельзя использовать как «верх», опорный вектор верха переключается на ось y.

GPU и CPU. Ядро CUDA запускается с фиксированной конфигурацией grid(32×32)×block(16×16) — 16 384 нити — и обходит пиксели кадра grid-stride циклами; конфигурация нитей не зависит от входных данных (требование курса).

Счетчик трассированных лучей накапливается через atomicAdd. CPU-версия использует те же функции трассировки и распараллеливается стандартными потоками std::thread по строкам изображения.

Описание программы

Программа — один файл kp.cu (около 1250 строк). Все структуры (vec3 с host/device-операторами, Scene, BodyG, Plane, Light, WireSeg, RenderParams, Hit) и ядро трассировки общие для версий CPU и GPU.

Ядро CUDA одно: renderKernel — рендер всех пикселей кадра с подсчетом лучей. Остальные функции: parseInput — разбор и валидация входа; loadTexture — загрузка текстуры пола (формат JP2); buildScene — генерация вершин, плоскостей граней, рёбер, реберных источников и каркаса; bodyHit, sceneHit, shadowVisible — пересечения лучей с телами и тени; shade — освещенность точки; transmitPath — под-путь преломления; traceRay — основной итеративный цикл трассировки; renderPixel — SSAA и тон-маппинг; makeRenderParams — камера; writeImage — запись кадра.

Обработка ошибок: все операции с файлами и вызовы CUDA проверяются; при ошибке печатается «ERROR: …» в stderr, и программа завершается с кодом возврата 0. Ограничения ввода: не более 100 000 кадров, стороны изображения до 16 384 пикселей, глубина трассировки до 64, SSAA до 64×64, до 256 источников на ребре, до 4 основных источников; символ «~» в путях раскрывается через переменную HOME.

Результаты

Программа собрана на ВМ и запущена с несколькими сериями: анимация 240 кадров 960×540 (SSAA 2×2, глубина 10), каркасная анимация 120 кадров, рендер в высоком разрешении 3840×2160 (SSAA 2×2, глубина 12), четыре статичных ракурса 1280×720 (SSAA 3×3) и бенчмарк GPU против CPU. Кадры конвертированы в PNG скриптом на Python, анимации собраны ffmpeg (24 кадра/с, libx264). Анимация считается в среднем за 169 мс на кадр (457 млн лучей на кадр); кадр 3840×2160 — 2.4 с и 8.9 млрд лучей.

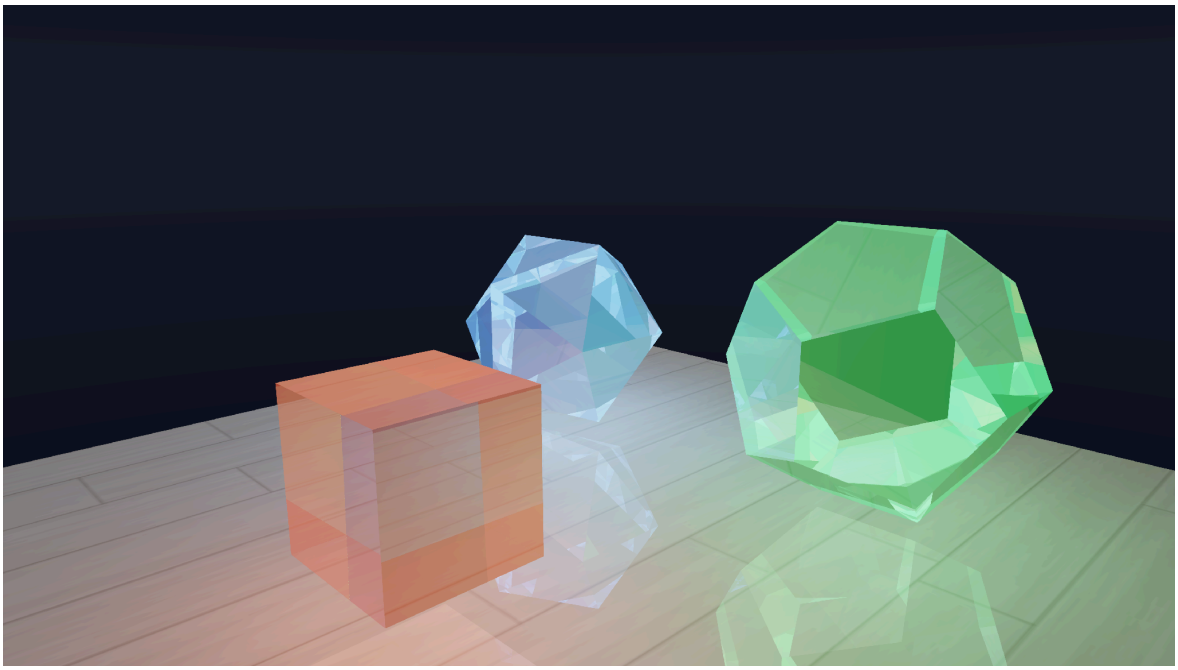


Рисунок 1 — Рендер в высоком разрешении (3840×2160, SSAA 2×2, глубина 12): гексаэдр, додекаэдр и икосаэдр на текстурированном полу; видны преломления, зеркальные отражения, эффект бесконечности внутри тел и свечение реберных источников

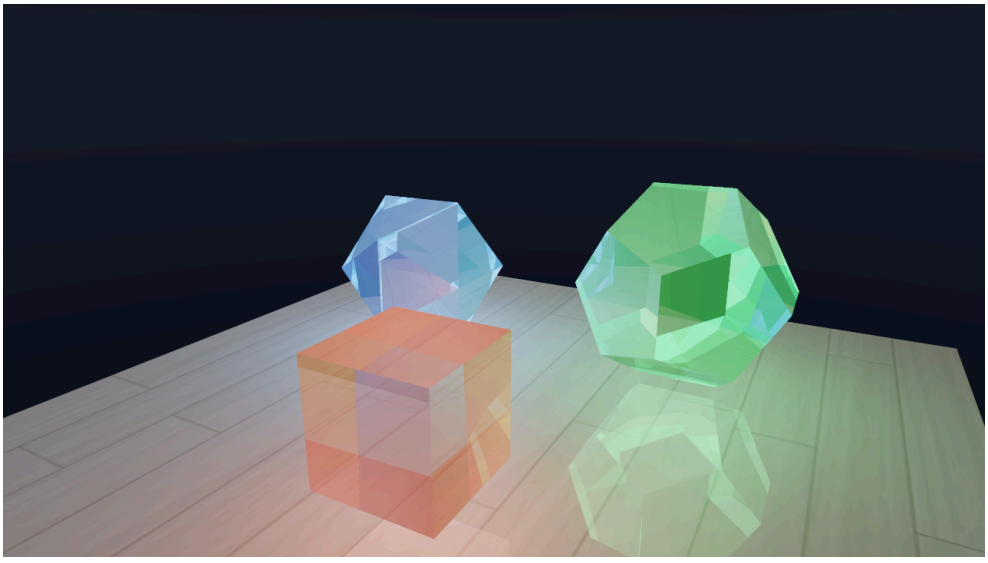


Рисунок 2 — Сцена с ракурса 1 (1280×720, SSAA 3×3)

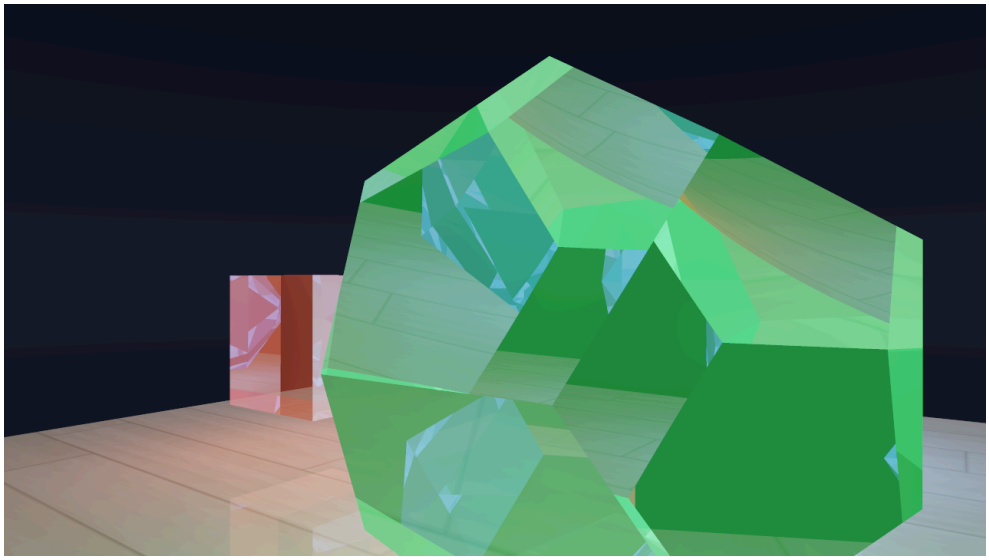


Рисунок 3 — Сцена с ракурса 2 (1280×720, SSAA 3×3)

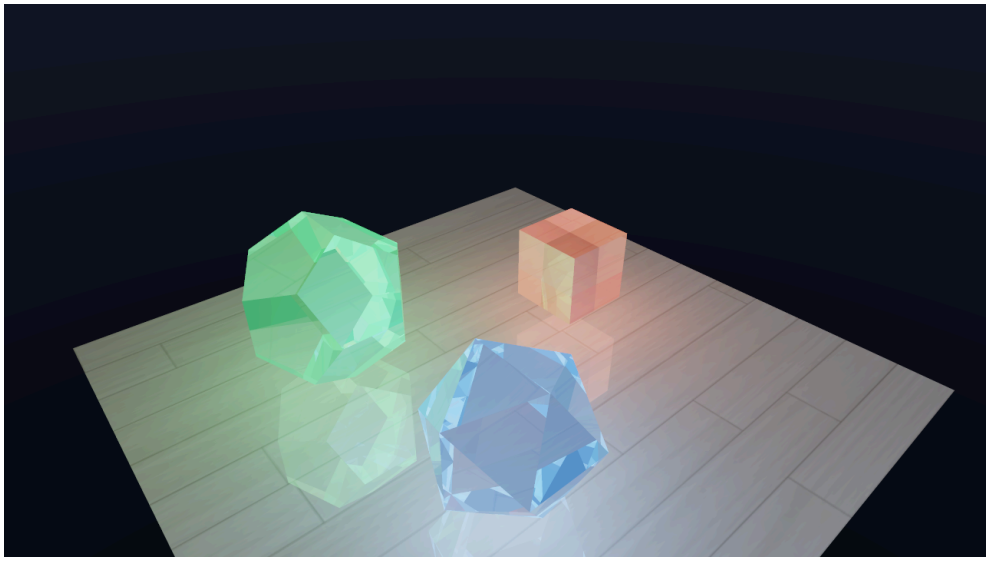


Рисунок 4 — Сцена с ракурса 3 (1280×720, SSAA 3×3)

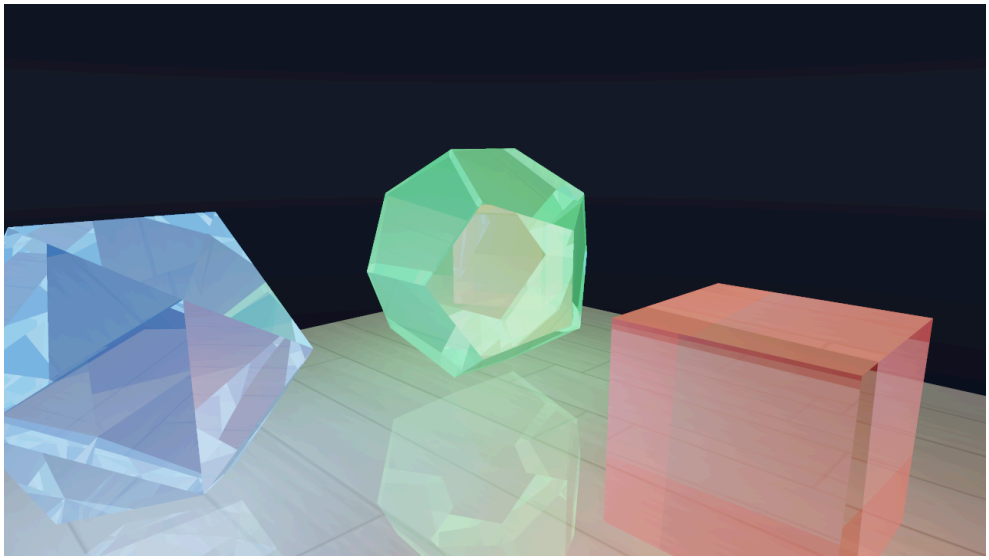


Рисунок 5 — Сцена с ракурса 4 (1280×720, SSAA 3×3)

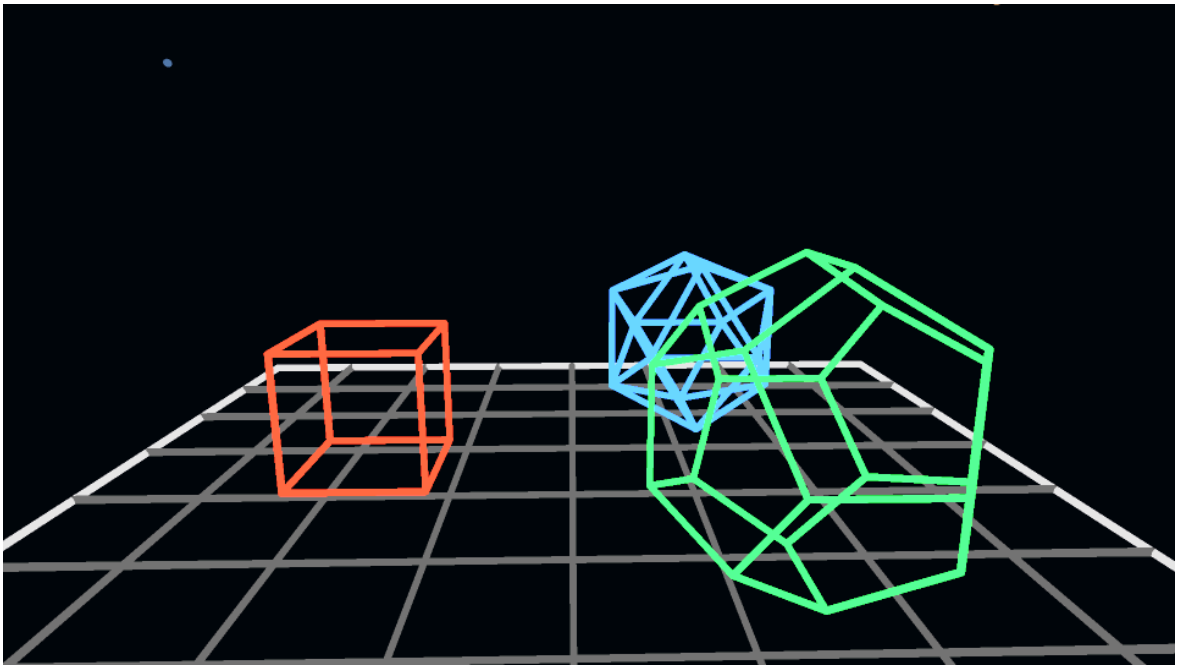


Рисунок 6 — Каркасный рендер (ключ --wireframe): рёбра тел, рамка и сетка пола, точечные источники на рёбрах и основные источники света

Таблица 1 — Сравнение производительности GPU и CPU (640×360, SSAA 1×1, глубина 8, 74 источника, 12 кадров)

Показатель	**GPU (CUDA)**	**CPU (std::thread)**
Конфигурация	grid(32×32)×block(16×16), 16 384 нити	2 потока std::thread

Среднее время кадра, мс	6.7	2696
Лучей на кадр	10.1 млн	10.1 млн
Ускорение	~403 раз	1

Бенчмарк (12 кадров 640×360, по одному лучу на пиксель, глубина 8): GPU считает кадр в среднем за 6.7 мс, CPU — за 2696 мс, ускорение около 403 раз; количество лучей на кадр (~10 млн, включая теневые) в обеих версиях одинаково — алгоритм детерминирован. Анимация (10 с при 24 кадрах в секунду) показывает облет сцены камерой, каркасная анимация — геометрию сцены и движение источников света по рёбрам тел.

Выводы

Главный технический результат — работающая обратная трассировка лучей на GPU: полупрозрачные полузеркальные платоновы тела с преломлением по Снеллиусу, итеративными отражениями и эффектом бесконечности за счет полного внутреннего отражения, текстурированный пол, до четырехсот источников света и SSAA. Самый тяжелый кадр — 3840×2160 с SSAA 2×2 — содержит 8.9 млрд трассированных лучей и считается за 2.4 с. Ядро с фиксированной конфигурацией нитей и grid-stride циклами дает ускорение около 403 раз против CPU-версии на тех же данных.

Проблема пересвета от суммарного HDR-освещения сотен источников решена стадией тон-маппинга (экспозиция + сжатие Рейнхарда): вместо жесткого ограничения диапазона, терявшего около трети кадра, изображение сжимается мягко, с сохранением цвета и деталей в ярких областях. Требование «без рекурсии» выполнено итеративным циклом с затухающим весом; прозрачность — упрощенная, с безпотерьным полным внутренним отражением внутри тела.